



Bikkad IT Institute

NumPy Tricks Notes

Compiled by Bikkad IT Institute | Teaching Notes

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

NumPy Tricks Notes

Sorting, Searching, Set Ops and Useful Functions

Prepared for Bikkad IT Institute Students | Level: Beginner to Intermediate | Environment: Jupyter Notebook or Google Colab

Topic Index

1. np.sort
2. np.append
3. np.concatenate
4. np.unique
5. np.expand_dims
6. np.where
7. np.argmax and np.argmin
8. np.cumsum and np.cumprod
9. np.percentile and np.median
10. np.histogram
11. np.corrcoef
12. np.isin
13. np.flip
14. np.put and np.delete
15. Set functions
16. np.clip

1. np.sort

Returns a sorted copy of an array, without changing the original.

Syntax

```
np.sort(a, axis=-1)
```

Example 1: Ascending sort of a 1D array

```
import numpy as np

np.random.seed(1)
a = np.random.randint(1, 100, 8)
print(a)

sorted_a = np.sort(a)
print(sorted_a)
```

Output / Result

```
[38 13 73 10 76  6 80 65]
[ 6 10 13 38 65 73 76 80]
```

Explanation

np.sort() returns a new array with the values arranged from smallest to largest by default; the original array a is left unchanged.

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Example 2: Descending sort

```
import numpy as np

np.random.seed(1)
a = np.random.randint(1, 100, 8)
print(a)

ascending = np.sort(a)
print(ascending)

descending = ascending[::-1]
print(descending)
```

Output / Result

```
[38 13 73 10 76  6 80 65]
[ 6 10 13 38 65 73 76 80]
[80 76 73 65 38 13 10  6]
```

Explanation

NumPy has no direct 'descending' argument, so the usual trick is to sort ascending and then reverse the result with `::-1`.

Example 3: Sorting a 2D array along an axis

```
import numpy as np

np.random.seed(1)
a = np.random.randint(1, 100, 8) # same draws as Example 1
print(a)

b = np.random.randint(1, 100, 12).reshape(3, 4)
print(b)

sorted_axis0 = np.sort(b, axis=0)
print(sorted_axis0)
```

Output / Result

```
[38 13 73 10 76  6 80 65]
[[17  2 77 72]
 [ 7 26 51 21]
 [19 85 12 29]]
[[ 7  2 12 21]
 [17 26 51 29]
 [19 85 77 72]]
```

Explanation

`axis=0` sorts each column independently (down the rows); `axis=1` (the default for 2D arrays) would sort each row independently instead.

2. np.append

Appends values along the given axis at the end of an array, returning a new array.

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Syntax

```
np.append(array, values, axis=None)
```

Example 1: Appending a single value

```
import numpy as np

a = np.array([10, 20, 30])
print(a)

appended = np.append(a, 200)
print(appended)
```

Output / Result

```
[10 20 30]
[ 10  20  30 200]
```

Explanation

Without an axis, the array is treated as flat and the new value is simply added at the end.

Example 2: Appending a new column to a 2D array

```
import numpy as np

np.random.seed(2)
b = np.arange(8).reshape(4, 2)
print(b)

extra_col = np.random.randint(1, 10, (b.shape[0], 1))
print(extra_col)

with_new_col = np.append(b, extra_col, axis=1)
print(with_new_col)
```

Output / Result

```
[[0 1]
 [2 3]
 [4 5]
 [6 7]]
[[9]
 [9]
 [7]
 [3]]
[[0 1 9]
 [2 3 9]
 [4 5 7]
 [6 7 3]]
```

Explanation

axis=1 appends the new values as an additional column, so the appended array must have the same number of rows as b.

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Example 3: Appending a new row to a 2D array

```
import numpy as np

np.random.seed(2)
b = np.arange(8).reshape(4, 2)
print(b)

new_row = np.array([[99, 99]])
print(new_row)

with_new_row = np.append(b, new_row, axis=0)
print(with_new_row)
```

Output / Result

```
[[0 1]
 [2 3]
 [4 5]
 [6 7]]
[[99 99]]
[[ 0  1]
 [ 2  3]
 [ 4  5]
 [ 6  7]
 [99 99]]
```

Explanation

axis=0 appends the new values as an additional row, so the appended array must have the same number of columns as b.

3. np.concatenate

Joins a sequence of arrays along an existing axis.

Syntax

```
np.concatenate((a, b), axis=0)
```

Example 1: Concatenating along axis 0 (rows)

```
import numpy as np

c = np.arange(6).reshape(2, 3)
print(c)

d = np.arange(6, 12).reshape(2, 3)
print(d)

stacked_rows = np.concatenate((c, d), axis=0)
print(stacked_rows)
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Output / Result

```
[[0 1 2]
 [3 4 5]]
[[ 6 7 8]
 [ 9 10 11]]
[[ 0 1 2]
 [ 3 4 5]
 [ 6 7 8]
 [ 9 10 11]]
```

Explanation

axis=0 stacks the arrays vertically, one below the other; both arrays must have the same number of columns.

Example 2: Concatenating along axis 1 (columns)

```
import numpy as np

c = np.arange(6).reshape(2, 3)
print(c)

d = np.arange(6, 12).reshape(2, 3)
print(d)

stacked_cols = np.concatenate((c, d), axis=1)
print(stacked_cols)
```

Output / Result

```
[[0 1 2]
 [3 4 5]]
[[ 6 7 8]
 [ 9 10 11]]
[[ 0 1 2 6 7 8]
 [ 3 4 5 9 10 11]]
```

Explanation

axis=1 joins the arrays side by side; both arrays must have the same number of rows.

Example 3: Concatenating flat 1D arrays

```
import numpy as np

e1 = np.array([1, 2, 3])
print(e1)

e2 = np.array([4, 5])
print(e2)

combined = np.concatenate((e1, e2))
print(combined)
```

Output / Result

```
[1 2 3]
[4 5]
[1 2 3 4 5]
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Explanation

For 1D arrays there is only one axis, so concatenate simply chains the values end to end.

4. np.unique

Finds the distinct (unique) values in an array, sorted in ascending order.

Syntax

```
np.unique(a, return_counts=False, return_index=False)
```

Example 1: Basic unique values

```
import numpy as np

arr = np.array([1, 1, 2, 2, 3, 3, 4, 4, 5])
print(arr)

unique_vals = np.unique(arr)
print(unique_vals)
```

Output / Result

```
[1 1 2 2 3 3 4 4 5]
[1 2 3 4 5]
```

Explanation

Duplicate values are collapsed and the remaining distinct values are returned in sorted order.

Example 2: Counting how often each value occurs

```
import numpy as np

arr = np.array([1, 1, 2, 2, 3, 3, 4, 4, 5])
print(arr)

values, counts = np.unique(arr, return_counts=True)
print(values)
print(counts)
```

Output / Result

```
[1 1 2 2 3 3 4 4 5]
[1 2 3 4 5]
[2 2 2 2 1]
```

Explanation

The second line is the unique values and the third is how many times each one appears: 1, 2, 3 and 4 appear twice each, and 5 appears once.

OUR LOCATIONS

Example 3: Locating the first occurrence of each value

```
import numpy as np

arr = np.array([1, 1, 2, 2, 3, 3, 4, 4, 5])
print(arr)

values, idx = np.unique(arr, return_index=True)
print(values)
print(idx)
```

Output / Result

```
[1 1 2 2 3 3 4 4 5]
[1 2 3 4 5]
[0 2 4 6 8]
```

Explanation

The second line is the unique values and the third is the position where each one first appears in arr: 1 at position 0, 2 at position 2, 3 at 4, 4 at 6 and 5 at 8.

5. np.expand_dims

Inserts a new axis of size 1 at the given position, increasing an array's dimension count.

Syntax

```
np.expand_dims(a, axis)
```

Example 1: Expanding a 1D array along axis 0

```
import numpy as np

a = np.array([1, 2, 3, 4])
print(a)
print(a.shape)

expanded_axis0 = np.expand_dims(a, axis=0)
print(expanded_axis0)
print(expanded_axis0.shape)
```

Output / Result

```
[1 2 3 4]
(4,)
[[1 2 3 4]]
(1, 4)
```

Explanation

The first two lines are a and its shape (4,). After expand_dims with axis=0, the array is printed as [[1 2 3 4]] and its shape is (1, 4) - a row vector with a new leading dimension.

OUR LOCATIONS

Example 2: Expanding along axis 1

```
import numpy as np

a = np.array([1, 2, 3, 4])
print(a)

expanded = np.expand_dims(a, axis=1)
print(expanded)
print(expanded.shape)
```

Output / Result

```
[1 2 3 4]
[[1]
 [2]
 [3]
 [4]]
(4, 1)
```

Explanation

axis=1 instead turns the same array into a (4,1) column vector, adding the new dimension at the end.

Example 3: Comparing both against the original

```
import numpy as np

a = np.array([1, 2, 3, 4])
print(a.shape)

shape_axis0 = np.expand_dims(a, axis=0).shape
print(shape_axis0)

shape_axis1 = np.expand_dims(a, axis=1).shape
print(shape_axis1)
```

Output / Result

```
(4, )
(1, 4)
(4, 1)
```

Explanation

expand_dims is commonly used to reshape a 1D array into a row or column vector before a broadcasting operation that needs an explicit 2D shape.

6. np.where

Returns the indices where a condition holds, or builds a new array by choosing between two values based on that condition.

Syntax

```
np.where(condition)
np.where(condition, value_if_true, value_if_false)
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Example 1: Finding indices where a condition holds

```
import numpy as np

np.random.seed(3)
a = np.random.randint(1, 100, 10)
print(a)

condition = a > 50
print(condition)

indices = np.where(condition)
print(indices)
```

Output / Result

```
[25  4 57 73  1 22 20 75 42 11]
[False False  True  True False False False  True False False]
(array([2, 3, 7]),)
```

Explanation

The second line is the condition $a > 50$. `np.where(condition)` returns a tuple of index arrays: positions 2, 3 and 7, where the values are 57, 73 and 75.

Example 2: Replacing values above a threshold

```
import numpy as np

np.random.seed(3)
a = np.random.randint(1, 100, 10)
print(a)

condition = a > 50
print(condition)

replaced = np.where(condition, 0, a)
print(replaced)
```

Output / Result

```
[25  4 57 73  1 22 20 75 42 11]
[False False  True  True False False False  True False False]
[25  4  0  0  1 22 20  0 42 11]
```

Explanation

With three arguments, `np.where` rebuilds the array: wherever the condition is True it uses the second argument (0), otherwise it keeps the original value.

OUR LOCATIONS

Example 3: Replacing every even value

```
import numpy as np

np.random.seed(3)
a = np.random.randint(1, 100, 10)
print(a)

is_even = a % 2 == 0
print(is_even)

replaced = np.where(is_even, 0, a)
print(replaced)
```

Output / Result

```
[25  4 57 73  1 22 20 75 42 11]
[False  True False False False  True  True False  True False]
[25  0 57 73  1  0  0 75  0 11]
```

Explanation

The condition can be any Boolean expression - here every even number is replaced with 0 while odd numbers are left untouched.

7. np.argmax and np.argmin

Return the index (position) of the maximum or minimum element, rather than the value itself.

Syntax

```
np.argmax(a, axis=None)
np.argmin(a, axis=None)
```

Example 1: Position of the largest and smallest value

```
import numpy as np

np.random.seed(3)
a = np.random.randint(1, 100, 10)
print(a)

max_index = np.argmax(a)
print(max_index)

min_index = np.argmin(a)
print(min_index)
```

Output / Result

```
[25  4 57 73  1 22 20 75 42 11]
7
4
```

Explanation

7 is the position of the largest value (75) and 4 is the position of the smallest value (1). argmax and argmin return where the extreme value is, which is often more useful than the value itself.

OUR LOCATIONS

Example 2: argmax along columns of a 2D array

```
import numpy as np

np.random.seed(3)
a = np.random.randint(1, 100, 10)
print(a)

b = np.random.randint(1, 100, 12).reshape(3, 4)
print(b)

col_argmax = np.argmax(b, axis=0)
print(col_argmax)
```

Output / Result

```
[25  4 57 73  1 22 20 75 42 11]
[[22 39 97 21]
 [45 94 40 15]
 [27 82 91 23]]
[1 1 0 2]
```

Explanation

With axis=0, each number is the row index of the largest value in that column: 45 is in row 1, 94 in row 1, 97 in row 0 and 23 in row 2 - giving [1 1 0 2].

Example 3: argmax along rows of a 2D array

```
import numpy as np

np.random.seed(3)
a = np.random.randint(1, 100, 10)
b = np.random.randint(1, 100, 12).reshape(3, 4)
print(b)

row_argmax = np.argmax(b, axis=1)
print(row_argmax)
```

Output / Result

```
[[22 39 97 21]
 [45 94 40 15]
 [27 82 91 23]]
[2 1 2]
```

Explanation

axis=1 instead returns, for each row, the column index of that row's largest value.

8. np.cumsum and np.cumprod

Compute a running (cumulative) sum or product of array elements, rather than a single total.

Syntax

OUR LOCATIONS

```
np.cumsum(a, axis=None)
np.cumprod(a, axis=None)
```

Example 1: Cumulative sum of a 1D array

```
import numpy as np

a = np.array([5, 10, 15, 20])
print(a)

running_total = np.cumsum(a)
print(running_total)
```

Output / Result

```
[ 5 10 15 20]
[ 5 15 30 50]
```

Explanation

Each output position holds the running total of all elements up to and including that position.

Example 2: Cumulative sum along an axis of a 2D array

```
import numpy as np

b = np.arange(1, 13).reshape(3, 4)
print(b)

running_total = np.cumsum(b, axis=1)
print(running_total)
```

Output / Result

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
[[ 1  3  6 10]
 [ 5 11 18 26]
 [ 9 19 30 42]]
```

Explanation

axis=1 restarts the running total at the beginning of each row; omitting axis would flatten the array first and cumulate over all elements.

Example 3: Cumulative product

```
import numpy as np

a = np.array([5, 10, 15, 20])
print(a)

running_product = np.cumprod(a)
print(running_product)
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Output / Result

```
[ 5 10 15 20]
[   5   50  750 15000]
```

Explanation

np.cumprod works exactly like cumsum but multiplies the running total instead of adding to it.

9. np.percentile and np.median

Summarize the relative position of values in a dataset; the median is simply the 50th percentile.

Syntax

```
np.percentile(a, q)
np.median(a)
```

Example 1: The 50th percentile

```
import numpy as np

np.random.seed(3)
a = np.random.randint(1, 100, 10)
print(a)

p50 = np.percentile(a, 50)
print(p50)
```

Output / Result

```
[25  4 57 73  1 22 20 75 42 11]
23.5
```

Explanation

With 10 values, the 50th percentile is the average of the two middle values after sorting (22 and 25), so it is 23.5 - exactly the median.

Example 2: Confirming it matches np.median

```
import numpy as np

np.random.seed(3)
a = np.random.randint(1, 100, 10)
print(a)

median_val = np.median(a)
print(median_val)
```

Output / Result

```
[25  4 57 73  1 22 20 75 42 11]
23.5
```

Explanation

`np.median(a)` is a convenience shortcut that always returns `np.percentile(a, 50)`.

Example 3: Multiple percentiles at once (quartiles)

```
import numpy as np

np.random.seed(3)
a = np.random.randint(1, 100, 10)
print(a)

quartiles = np.percentile(a, [25, 50, 75])
print(quartiles)
```

Output / Result

```
[25  4 57 73  1 22 20 75 42 11]
[13.25 23.5  53.25]
```

Explanation

Passing a list of percentages computes several percentiles in a single call, commonly used to get the quartiles of a dataset.

10. np.histogram

Counts how many values fall into each of a set of bins, the numeric basis behind a histogram chart.

Syntax

```
np.histogram(a, bins)
```

Example 1: Two wide bins

```
import numpy as np

np.random.seed(3)
a = np.random.randint(1, 100, 10)
print(a)

counts, edges = np.histogram(a, bins=[0, 50, 100])
print(counts)
print(edges)
```

Output / Result

```
[25  4 57 73  1 22 20 75 42 11]
[7 3]
[  0  50 100]
```

Explanation

The second line is counts: 7 values fall in 0-50 and 3 values in 50-100. The last line is the bin edges we passed in.

OUR LOCATIONS

Example 2: More, evenly-sized bins

```
import numpy as np

np.random.seed(3)
a = np.random.randint(1, 100, 10)
print(a)

counts, edges = np.histogram(a, bins=[0, 25, 50, 75, 100])
print(counts)
print(edges)
```

Output / Result

```
[25  4 57 73  1 22 20 75 42 11]
[5 2 2 1]
[  0 25 50 75 100]
```

Explanation

With the edges 0, 25, 50, 75, 100, the counts are 5 values in 0-25, 2 in 25-50, 2 in 50-75 and 1 in 75-100. More edges give a finer breakdown - exactly what a histogram chart shows as bars.

Example 3: Automatic equal-width bins

```
import numpy as np

np.random.seed(3)
a = np.random.randint(1, 100, 10)
print(a)

counts, edges = np.histogram(a, bins=4)
print(counts)

rounded_edges = np.round(edges, 1)
print(rounded_edges)
```

Output / Result

```
[25  4 57 73  1 22 20 75 42 11]
[3 3 1 3]
[ 1. 19.5 38. 56.5 75. ]
```

Explanation

`bins=4` asks NumPy to split the range from the minimum (1) to the maximum (75) into 4 equal-width bins. The second line is the count in each bin and the last line is the 5 automatic edges, rounded to 1 decimal.

11. np.corrcoef

Returns the Pearson correlation coefficient matrix, measuring how strongly two variables move together.

Syntax

```
np.corrcoef(x, y)
```

OUR LOCATIONS

Example 1: A strong positive correlation

```
import numpy as np

salary = np.array([20000, 30000, 40000, 50000, 60000])
print(salary)

experience = np.array([1, 2, 3, 5, 4])
print(experience)

corr_matrix = np.corrcoef(salary, experience)
print(corr_matrix)

rounded_corr = np.round(corr_matrix, 3)
print(rounded_corr)
```

Output / Result

```
[20000 30000 40000 50000 60000]
[1 2 3 5 4]
[[1.  0.9]
 [0.9 1.  ]]
[[1.  0.9]
 [0.9 1.  ]]
```

Explanation

The third output is the full correlation matrix and the last is the same matrix rounded to 3 decimals. The off-diagonal value 0.9 is the correlation between salary and experience - close to 1, so they tend to increase together.

Example 2: A near-zero correlation

```
import numpy as np

np.random.seed(4)
x = np.arange(10)
print(x)

y = np.random.randint(1, 100, 10)
print(y)

corr_matrix = np.corrcoef(x, y)
print(corr_matrix)

rounded_corr = np.round(corr_matrix, 3)
print(rounded_corr)
```

Output / Result

```
[0 1 2 3 4 5 6 7 8 9]
[47 56 70  2 88 73 51 10 59 95]
[[1.          0.17358256]
 [0.17358256 1.          ]]
[[1.    0.174]
 [0.174 1.    ]]
```

Explanation

The off-diagonal value, about 0.17, is close to 0 because the random y values are essentially unrelated to x. The last output is the same matrix rounded to 3 decimals.

OUR LOCATIONS

Example 3: Every array perfectly correlates with itself

```
import numpy as np

salary = np.array([20000, 30000, 40000, 50000, 60000])
print(salary)

self_corr = np.corrcoef(salary, salary)
print(self_corr)
```

Output / Result

```
[20000 30000 40000 50000 60000]
[[1.  1.]
 [1.  1.]]
```

Explanation

The diagonal of any correlation matrix is always 1, since a variable is always perfectly correlated with itself.

12. np.isin

Checks, element by element, whether values from one array also appear in another array.

Syntax

```
np.isin(a, test_values)
```

Example 1: Keeping only values from a target list

```
import numpy as np

a = np.array([15, 20, 35, 40, 55, 60, 75, 80])
print(a)

items = [20, 40, 60, 80, 100]
print(items)

mask = np.isin(a, items)
print(mask)

filtered = a[mask]
print(filtered)
```

Output / Result

```
[15 20 35 40 55 60 75 80]
[20, 40, 60, 80, 100]
[False  True False  True False  True False  True]
[20 40 60 80]
```

Explanation

np.isin builds a Boolean mask marking every element of a that also occurs in items, which can then be used to filter a directly.

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Example 2: Excluding values with the negation

```
import numpy as np

a = np.array([15, 20, 35, 40, 55, 60, 75, 80])
print(a)

items = [20, 40, 60, 80, 100]
mask = np.isin(a, items)
print(mask)

negated_mask = ~mask
print(negated_mask)

excluded = a[negated_mask]
print(excluded)
```

Output / Result

```
[15 20 35 40 55 60 75 80]
[False True False True False True False True]
[ True False  True False  True False  True False]
[15 35 55 75]
```

Explanation

Prefixing the mask with ~ flips True and False, so this keeps every element that is NOT found in items.

Example 3: Using isin on a 2D array

```
import numpy as np

b = np.arange(12).reshape(3, 4)
print(b)

target = [0, 5, 10]
print(target)

mask = np.isin(b, target)
print(mask)
```

Output / Result

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[0, 5, 10]
[[ True False False False]
 [False  True False False]
 [False False  True False]]
```

Explanation

isin works on arrays of any shape; the result is a same-shaped Boolean array rather than a flat list.

13. np.flip

Reverses the order of array elements along the given axis while preserving the array's shape.

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Syntax

```
np.flip(a, axis=None)
```

Example 1: Flipping a 1D array

```
import numpy as np

a = np.array([10, 20, 30, 40, 50])
print(a)

flipped = np.flip(a)
print(flipped)
```

Output / Result

```
[10 20 30 40 50]
[50 40 30 20 10]
```

Explanation

Without an axis, flip reverses the entire flattened sequence of elements, similar to `a[::-1]`.

Example 2: Flipping a 2D array along columns

```
import numpy as np

b = np.arange(12).reshape(3, 4)
print(b)

flipped_cols = np.flip(b, axis=1)
print(flipped_cols)
```

Output / Result

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[[ 3  2  1  0]
 [ 7  6  5  4]
 [11 10  9  8]]
```

Explanation

`axis=1` reverses the order of columns within every row, while the row order itself stays the same.

Example 3: Flipping along rows instead

```
import numpy as np

b = np.arange(12).reshape(3, 4)
print(b)

flipped_rows = np.flip(b, axis=0)
print(flipped_rows)
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Output / Result

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[[ 8  9 10 11]
 [ 4  5  6  7]
 [ 0  1  2  3]]
```

Explanation

axis=0 instead reverses the order of the rows, while each row's own contents stay in the same order.

14. np.put and np.delete

np.put replaces specific elements of an array in place; np.delete returns a new array with certain elements removed.

Syntax

```
np.put(a, indices, values)
np.delete(a, indices, axis=None)
```

Example 1: Replacing elements in place with np.put

```
import numpy as np

a = np.array([10, 20, 30, 40, 50])
print(a)

np.put(a, [0, 1], [110, 530])
print(a)
```

Output / Result

```
[10 20 30 40 50]
[110 530 30 40 50]
```

Explanation

np.put treats the array as flattened and directly overwrites the values at the given positions - unlike most NumPy functions, it modifies a in place rather than returning a new array.

Example 2: Deleting elements from a 1D array

```
import numpy as np

b = np.array([10, 20, 30, 40, 50])
print(b)

after_delete = np.delete(b, [0, 2, 4])
print(after_delete)
```

Output / Result

```
[10 20 30 40 50]
[20 40]
```

Explanation

np.delete returns a new array with the elements at the given positions removed, leaving the original array b untouched.

Example 3: Deleting a whole row from a 2D array

```
import numpy as np

c = np.arange(12).reshape(3, 4)
print(c)

after_delete = np.delete(c, 1, axis=0)
print(after_delete)
```

Output / Result

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[[ 0  1  2  3]
 [ 8  9 10 11]]
```

Explanation

Passing axis=0 removes an entire row (here, row index 1) instead of individual flattened elements.

15. Set functions

NumPy treats 1D arrays as mathematical sets, offering union, intersection, difference, and membership operations.

Syntax

```
np.union1d(a, b); np.intersect1d(a, b)
np.setdiff1d(a, b); np.setxor1d(a, b)
```

Example 1: Union and intersection

```
import numpy as np

m = np.array([1, 2, 3, 4, 5])
print(m)

n = np.array([3, 4, 5, 6, 7])
print(n)

union = np.union1d(m, n)
print(union)

intersection = np.intersect1d(m, n)
print(intersection)
```

OUR LOCATIONS

Output / Result

```
[1 2 3 4 5]
[3 4 5 6 7]
[1 2 3 4 5 6 7]
[3 4 5]
```

Explanation

Union keeps every distinct value from both arrays; intersection keeps only the values common to both.

Example 2: Difference and symmetric difference

```
import numpy as np

m = np.array([1, 2, 3, 4, 5])
print(m)

n = np.array([3, 4, 5, 6, 7])
print(n)

difference = np.setdiff1d(n, m)
print(difference)

sym_diff = np.setxor1d(m, n)
print(sym_diff)
```

Output / Result

```
[1 2 3 4 5]
[3 4 5 6 7]
[6 7]
[1 2 6 7]
```

Explanation

setdiff1d(n, m) keeps values in n that are not in m; setxor1d keeps values present in exactly one of the two arrays.

Example 3: Checking membership with in1d

```
import numpy as np

m = np.array([1, 2, 3, 4, 5])
print(m)

mask = np.isin(m, [1])
print(mask)

filtered = m[mask]
print(filtered)
```

Output / Result

```
[1 2 3 4 5]
[ True False False False False]
[1]
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Explanation

Filtering with a membership test (the modern equivalent of the older `np.in1d`) picks out only the elements of `m` that match a given value or list of values.

16. np.clip

Limits (clips) every value in an array to stay within a given minimum and maximum range.

Syntax

```
np.clip(a, a_min, a_max)
```

Example 1: Clipping to both a minimum and maximum

```
import numpy as np

np.random.seed(3)
a = np.random.randint(1, 100, 10)
print(a)

clipped = np.clip(a, a_min=25, a_max=75)
print(clipped)
```

Output / Result

```
[25  4 57 73  1 22 20 75 42 11]
[25 25 57 73 25 25 25 75 42 25]
```

Explanation

Every value below 25 is raised to 25, every value above 75 is lowered to 75, and values already inside the range are left unchanged.

Example 2: Clipping with only a lower bound

```
import numpy as np

np.random.seed(3)
a = np.random.randint(1, 100, 10)
print(a)

clipped_lower = np.clip(a, a_min=50, a_max=None)
print(clipped_lower)
```

Output / Result

```
[25  4 57 73  1 22 20 75 42 11]
[50 50 57 73 50 50 50 75 50 50]
```

Explanation

Passing `a_max=None` leaves the upper end unrestricted, so this only raises values below 50 up to 50.

OUR LOCATIONS

Example 3: Clipping with only an upper bound

```
import numpy as np

np.random.seed(3)
a = np.random.randint(1, 100, 10)
print(a)

clipped_upper = np.clip(a, a_min=None, a_max=50)
print(clipped_upper)
```

Output / Result

```
[25  4 57 73  1 22 20 75 42 11]
[25  4 50 50  1 22 20 50 42 11]
```

Explanation

Passing `a_min=None` leaves the lower end unrestricted, so this only lowers values above 50 down to 50.

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune